

TRACE 详细设计说明书

1. 文档说明

1.1 文档目的

本文档说明 TRACE 当前交付版本的详细设计，包括系统架构、部署结构、后端模块、前端模块、核心业务流程、状态机、执行器、评测链路、接口设计、异常处理、安全边界和交付运行方式。

1.2 当前交付边界

当前已实现：

- FastAPI 后端 API；
- SQLAlchemy + Alembic 数据模型和迁移；
- PostgreSQL 主存储；
- Redis + Celery 异步任务；
- Vue 3 + TypeScript 前端；
- 本地项目、快照、运行配置、策略版本、测试计划和测试运行；
- Agent 测试生成、pytest 执行、一次受限 Reflection；
- trace step、run event、pytest case result、artifact 和 run-level report；
- eval dataset、seeded bug、bug variant、experiment、clean run、replay 和 metrics；
- workspace cleanup / replay cache cleanup；
- Docker Compose 产品栈交付。

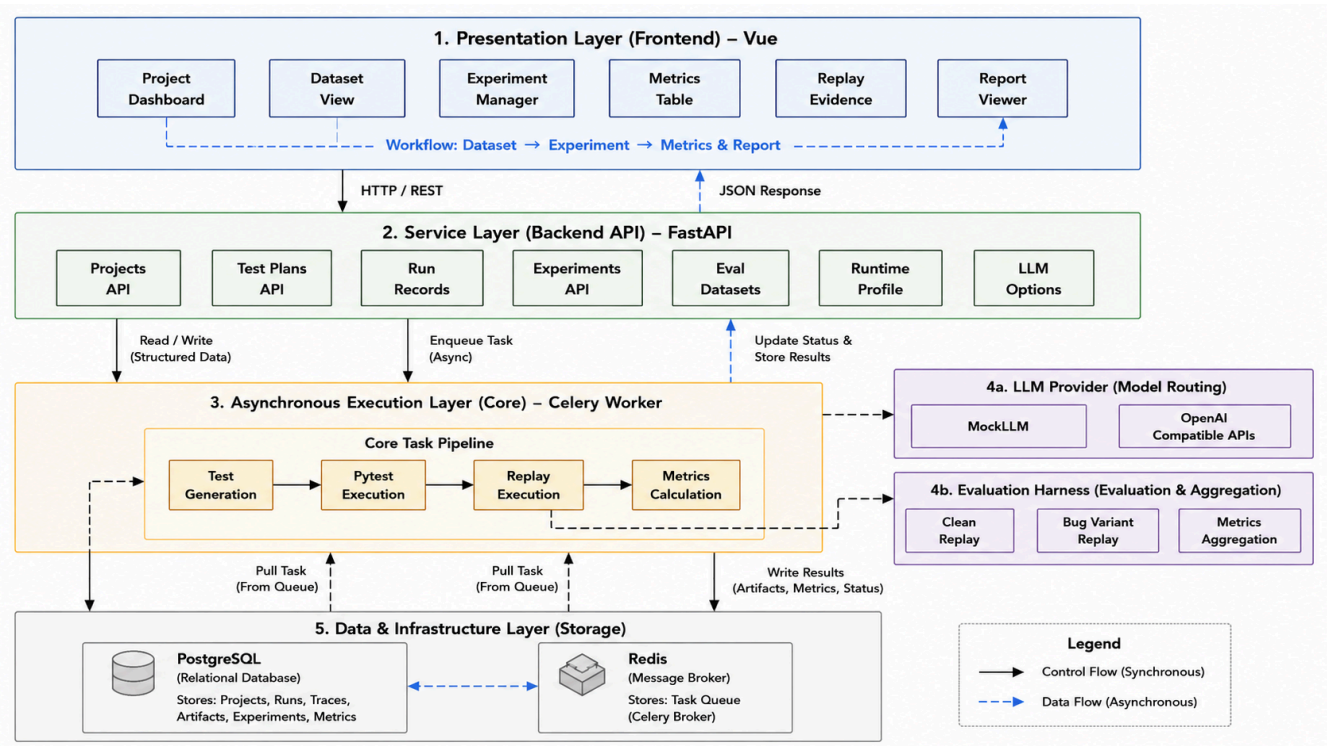
当前不承诺：

- 远程 Runner 协议；
- 分布式调度；
- MinIO/S3 artifact storage；
- 完整 OpenTelemetry；
- Docker executor 默认交付；
- 完整安全沙箱；
- experiment-level Markdown 报告服务。

2. 总体架构设计

2.1 架构分层

TRACE 当前采用前后端分离 + 异步 Worker + 关系数据库的结构。



实验写入-1.png

主要分层：

分层	代码位置	职责
前端展示层	frontend/src	页面路由、表单、运行详情、实验详情、策略对比
API 层	backend/app/api	HTTP 接口、请求校验、错误映射
Service 层	backend/app/services	业务规则、状态流转、实验编排、指标聚合
Repository 层	backend/app/repositories	数据访问封装
Model 层	backend/app/models	SQLAlchemy ORM 模型
Schema 层	backend/app/schemas	API、工具、trace、评测契约
Agent 层	backend/app/agents	策略、LLM、提示词、报告、契约守卫
Tool 层	backend/app/tools	Agent 可调用工具和 pytest 执行器
Worker 层	backend/app/workers	Celery app 与异步任务入口
Eval 层	eval	demo 数据、评测 harness、指标计算辅助

2.2 Control Plane 与 Execution Plane

当前实现中，control plane 和 execution plane 的边界如下：

- Control plane: API、service、repository、database、frontend。
- Execution plane: Worker、Agent runtime、tool registry、pytest executor、experiment workspace。

API 层不直接执行耗时的 Agent 任务。普通 run 和 experiment 都先落库，再由 Celery Worker 根据 ID 回库恢复上下文并执行。这个设计减少队列消息体大小，也避免把临时对象塞进 Redis。

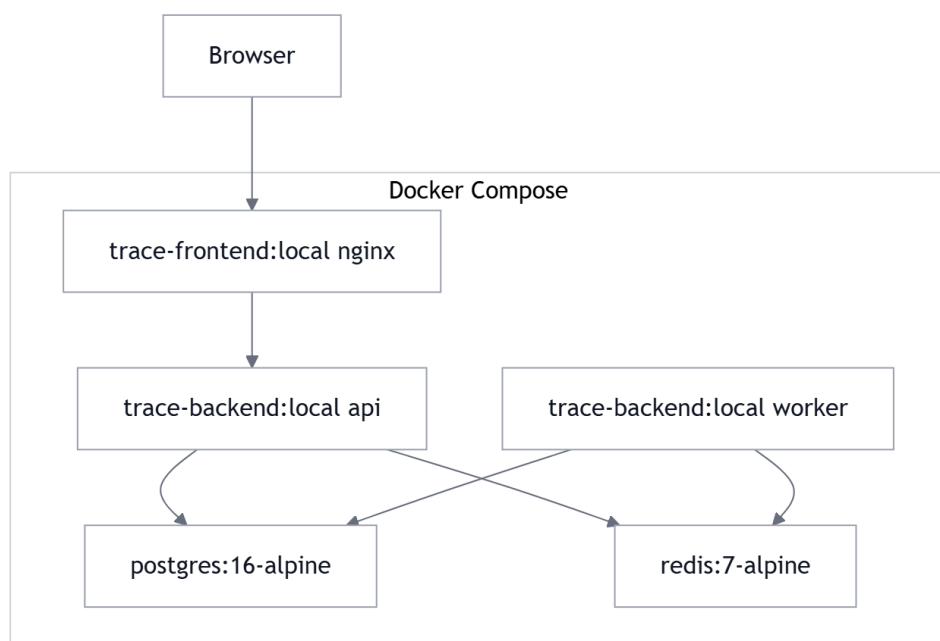
2.3 关键设计原则

1. 运行输入冻结：run 创建时冻结 runtime snapshot 和 strategy snapshot。
2. 过程先落库：状态变化前先记录 run event 或 trace step。
3. 工具受控：Agent 只能通过 ToolRegistry 调用白名单工具。
4. 测试真实执行：pytest 通过执行器运行，结果解析为结构化 case result。
5. Reflection 有红线：只允许修复生成测试，不允许修业务代码或削弱断言。
6. 评测防污染：replay 使用 frozen test set，不再调用 LLM。
7. 指标不造假：invalid/system failure 不压成 0 捕获。
8. 安全边界诚实：local subprocess 不是沙箱，Docker Compose 不是 Docker executor。

3. 部署设计

3.1 Docker Compose 交付部署

Docker Compose 是推荐交付路径。



部署方式.png

Compose 服务：

服务	镜像	端口	职责
postgres	postgres:16-alpine	5432	主数据库
redis	redis:7-alpine	6379	Celery broker/result backend
api	trace-backend:local	8000	FastAPI 后端
worker	trace-backend:local	无外部端口	Celery Worker
frontend	trace-frontend:local	5186 → 80	nginx 托管前端静态资源

Compose 启动时 API 容器负责：

1. 等待 PostgreSQL / Redis ready；
2. 初始化数据库；
3. seed strategy versions；
4. seed demo eval dataset；
5. 启动 API 服务。

3.2 本地开发部署

本地开发通常使用：

- PostgreSQL / Redis：Docker Compose 启动基础服务；
- API：本机 Python 环境执行 `backend/scripts/run_api.py` ；
- Worker：本机 Python 环境执行 `backend/scripts/run_worker.py` ；
- Frontend： `frontend/npm run dev` 。

本地开发适合调试日志和单步验证；Compose 适合最终交付演示。

3.3 存储设计

当前存储分三类：

类型	位置	内容
主业务数据	PostgreSQL	projects、runs、experiments、metrics、trace、pytest results
短期队列状态	Redis	Celery 任务分发和结果状态
文件型产物	本地文件系统	generated test set、pytest artifacts、experiment workspace

目前未接入对象存储。 `run_artifacts` 保存的是 artifact 元数据和 URI，不直接承载大文件内容。

4. 后端应用设计

4.1 应用入口

后端入口位于：

- backend/app/main.py
- backend/app/api/router.py

create_app() 完成：

1. 加载 ORM 模型；
2. 可选 seed strategy versions；
3. 创建 FastAPI app；
4. 注册 API router；
5. 暴露 /healthz 。

/healthz 返回：

- ok=true ；
- 当前 LLM 配置的 provider、model、base_url 摘要。

注意： /healthz 不回显 API key。

4.2 API Router 组织

backend/app/api/router.py 统一 include 以下路由：

路由模块	职责
projects.py	项目、快照、项目下计划和运行查询
test_plans.py	测试计划创建与详情
test_runs.py	run 创建、入队、取消、重试、结果查询
versioning.py	strategy、prompt version、tool schema version
strategy_versions.py	strategy version 单独查询
runtime_profiles.py	runtime profile、executor status、preflight
evaluation.py	dataset、task、seeded bug、variant、mutation discovery
experiments.py	experiment 创建、启动、进度、metrics、cleanup
llm_options.py	LLM option 查询
local.py	本地目录选择辅助

API 层只做 HTTP 语义、异常映射和 Pydantic response model 约束；业务判断在 service 层。

4.3 Service 层设计

Service 层是当前系统的业务核心。

Service	职责
projects.py	项目与快照创建、查询
test_plans.py	测试计划创建
test_runs.py	run 创建、入队、同步执行、取消、重试、快照冻结
strategies.py	seed 默认策略版本
runtime_profiles.py	runtime profile 校验、preflight、默认配置
evaluation.py	dataset/task/bug/variant 管理与 readiness
experiments.py	experiment 状态机、clean run、replay 编排
experiment_metrics.py	指标聚合、capture matrix、runtime execution summary
evaluation_events.py	EvaluationEvent 投影与 prompt backfeed 边界
replay_scheduler.py	replay 并发、重试调度
replay_orchestration.py	variant replay job 构造和结果持久化
replay_cache.py	replay cache 清理
artifact_cleanup.py	workspace cleanup 和 artifact inventory
source_context*.py	源码上下文收集与完整性检查
path_policy.py	snapshot root 与生成测试目录校验
llm_options.py	LLM option 汇总和脱敏

Service 层的几个硬规则：

- run 创建不执行 Agent；
- enqueue 只投递 run ID；
- worker 执行时重新读取 DB；
- experiment 只有 draft 可原地启动；
- failed experiment 不允许原地 rerun；
- clean run 失败和 invalid test set 必须分类处理；
- replay 不调用 LLM。

5. 后端分层详细设计

5.1 Repository 层

Repository 位于 `backend/app/repositories` , 用于封装常用查询和新增操作。

设计目的:

- 减少 service 中重复 SQLAlchemy 查询;
- 保持 API 层不直接碰 ORM 细节;
- 让测试更容易针对 service 行为断言。

当前 repository 覆盖:

- projects;
- test plans;
- test runs;
- strategies;
- versioning;
- runtime profiles;
- evaluation。

5.2 ORM Model 层

Model 位于 `backend/app/models` , 按业务域拆分:

文件	表
project.py	projects 、 project_snapshots
runtime_profile.py	runtime_profiles
strategy.py	strategy_versions
versioning.py	strategies 、 prompt_versions 、 tool_schema_versions
test_plan.py	test_plans
test_run.py	test_runs 、 run_plan_items 、 run_attempts
trace.py	trace_steps 、 run_events
artifacts.py	generated_test_files 、 generated_test_cases 、 pytest_case_results 、 run_artifacts 、 test_reports
evaluation.py	eval_datasets 、 eval_tasks 、 seeded_bugs 、 bug_variants 、 experiments 、 experiment_clean_runs 、 test_replays 等

Model 层不写复杂业务逻辑。复杂状态流转放到 service。

5.3 Schema 层

Schema 位于 `backend/app/schemas`。

主要类型：

- API request / response schema;
- Agent record schema;
- Tool input / output schema;
- Trace schema;
- Evaluation contract;
- Runtime snapshot contract;
- Strategy snapshot contract。

Schema 层承担两个职责：

1. 对外 API 的数据契约;
2. 内部模块之间的结构化边界。

5.4 Recorder 适配层

`backend/app/recorders/sqlalchemy_recorder.py` 是 Agent 层和数据库之间的关键适配。

Agent 运行时只面向 `RunRecorder` 协议写入：

- run;
- run event;
- trace step;
- plan item;
- attempt;
- generated file;
- generated case;
- pytest result;
- artifact;
- report。

SQLAlchemy recorder 将这些 record 转换成 ORM 表记录并提交。

设计收益：

- Agent 策略不直接依赖 SQLAlchemy;
- 未来可替换 recorder 或调整事务边界;
- 运行过程可以边执行边可见;

- 失败时不会只剩最终状态，没有过程证据。

6. 异步任务设计

6.1 Celery 配置

Worker 位于：

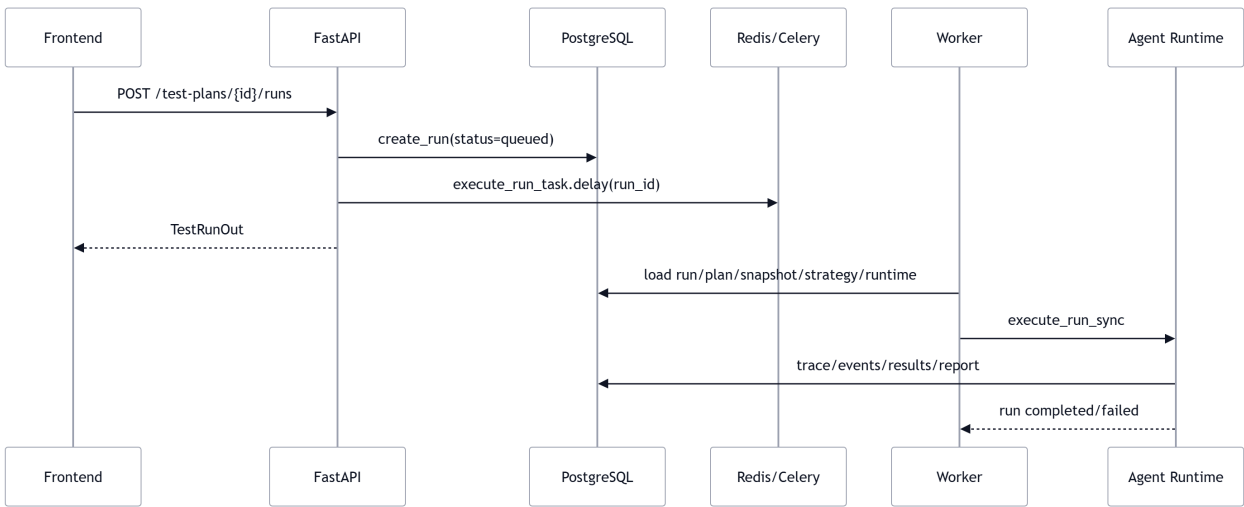
- backend/app/workers/celery_app.py
- backend/app/workers/tasks.py

任务包括：

任务名	函数	作用
trace.execute_run	execute_run_task	执行普通 test run
trace.execute_experiment	execute_experiment_task	执行 experiment

任务入参只保留 ID 和少量覆盖参数。

6.2 普通 Run 异步执行



异步执行.png

execute_run_task 发生异常时会重新读取 run，确保状态已经由 service 持久化；如果 run 不存在才继续抛出异常。

6.3 Experiment 异步执行

Experiment 创建后是 draft。启动时：

1. API 调用 `enqueue_experiment` ;
2. service 将 `draft` 改为 `queued`;
3. 投递 `execute_experiment_task` ;
4. Worker 调用 `run_experiment` ;
5. experiment 进入 `running`;
6. 完成后进入 `completed`, 异常进入 `failed`, 取消进入 `cancelled`。

Experiment 不支持 `failed/completed` 原地 rerun。这个约束是为了避免旧 metrics 和新 replay 混在同一 experiment 下。

7. 普通 Test Run 设计

7.1 创建阶段

`create_run()` 的主要步骤:

1. 校验 test plan 存在;
2. 校验 snapshot 存在且属于 plan 的 project;
3. 校验 project 存在;
4. 校验 snapshot root 在 project root 范围内;
5. 解析 strategy version;
6. 解析 LLM 配置或 `llm override`;
7. 获取 runtime profile, 缺省时创建 `Default Local Subprocess` ;
8. 构造 runtime snapshot;
9. 构造 strategy snapshot;
10. 创建 `test_runs` 记录, status 为 `queued`。

7.2 Runtime Snapshot

Runtime snapshot 来自 runtime profile 和 run 创建时的 budget/output override。

包含:

- runtime profile ID 和名称;
- executor;
- image;
- working dir;
- Python version;
- install command;

- test command;
- network policy;
- timeout seconds;
- env template;
- resource limits;
- replay policy;
- artifact policy;
- cleanup policy;
- executor capabilities;
- budget override;
- output options.

env template 会经过脱敏处理, secret 不进入 snapshot。

7.3 Strategy Snapshot

Strategy snapshot 来自 strategy version、prompt version、tool schema version 和 LLM 解析结果。

包含:

- strategy version ID;
- prompt version ID、content hash、content;
- tool schema version ID、content hash、schema JSON;
- workflow type;
- allow reflection;
- resolved LLM provider、model、temperature、max output tokens、model params;
- repeat index;
- secret included 标记固定为 false;
- 兼容旧接口的扁平字段。

设计红线: API key 不进入 snapshot。

7.4 执行阶段

`execute_run_sync()` 是普通 run 的实际执行入口。

执行步骤:

1. 加载 run;
2. 已 completed/failed/cancelled 则直接返回;

3. 加载 plan、snapshot、strategy、project;
4. 校验 snapshot root;
5. 创建或确认 tests/generated 生成测试目录;
6. 构造 SQLAlchemyRunRecorder;
7. 从 run. strategy_snapshot 重建 StrategyVersionSpec;
8. 选择 LLM;
9. 构造 ToolContext;
10. 调用 Agent execute_run() ;
11. 关闭 LLM client。

7.5 状态机

普通 run 状态字段:

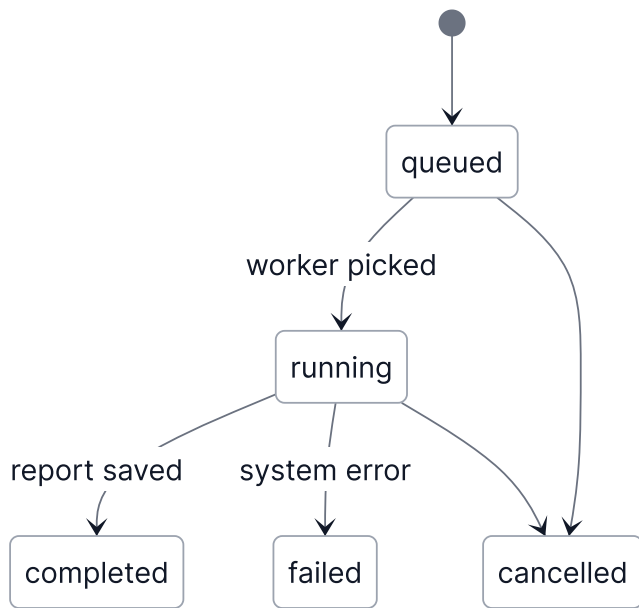
- status : 外部状态;
- stage : running 内部阶段。

状态取值:

外部状态	含义
queued	已创建或已入队
running	Worker 正在执行
completed	系统流程完成, 可能包含 pytest 失败
failed	系统流程失败
cancelled	用户取消

stage 取值:

- preparing;
- analyzing;
- planning;
- generating;
- executing;
- reflecting;
- reexecuting;
- summarizing。



stage 在 `running` 内细分, `completed` 后 stage 置空。失败时 stage 保留失败发生位置。

7.6 Retry 与 Cancel

retry 设计:

- 不在原 run 下追加 attempt;
- 克隆一条新 `test_runs` ;
- 新 run 的 `retry_of_run_id` 指向原 run;
- 新 run 重新入队。

cancel 设计:

- 软取消;
- 对 `queued/running` run 标记 `cancelled`;
- 记录 run event;
- 不承诺进程级强杀。

8. Agent 运行设计

8.1 Agent Runtime

Agent 核心位于:

- `backend/app/agents/orchestrator.py`
- `backend/app/agents/runtime.py`

- backend/app/agents/strategies
- backend/app/agents/prompts.py
- backend/app/agents/structured.py
- backend/app/agents/report.py

execute_run() 负责：

1. 保存 queued run;
2. 构造 AgentContext;
3. 设置 preparing;
4. 根据 workflow type 选择策略;
5. 执行策略;
6. 进入 summarizing;
7. 构建 report;
8. 保存 report;
9. 标记 completed;
10. 异常时先写 run event 和 trace, 再标记 failed。

8.2 Strategy 类型

当前策略包括：

策略	文件	行为
Direct	direct.py	分析后直接生成测试并执行
Plan-and-Execute	plan_execute.py	先规划多个 plan item, 再逐项生成执行
ReAct + Reflection	react_reflection.py	分析、生成、执行, 并在失败时允许反思

策略通过 get_strategy(workflow_type) 选择。

8.3 公共执行逻辑

strategies/common.py 封装策略共用步骤：

- do_analyze()：调用 analyze_project;
- make_attempt()：创建 run attempt;
- do_generate()：调用 LLM 生成测试;
- do_run()：调用 run_pytest;
- execute_item()：生成、执行和可选反思;
- _reflect_once()：一次 Reflection;

- `_register_cases()` : 解析 generated test case;
- `_map_results()` : pytest result 与 generated case 映射。

8.4 Structured LLM 输出

LLM 输出通过 `generate_structured()` 提取和校验 JSON。

设计目的:

- 防止自由文本直接进入执行流程;
- 支持非法 JSON 重试;
- 将 raw response、错误和结构化结果纳入 trace;
- 让策略行为可回放。

8.5 Prompt 设计

`prompts.py` 生成:

- system prompt;
- plan messages;
- generate messages;
- reflect messages;
- generate retry messages;
- prompt bundle。

prompt bundle 会写入 strategy snapshot, 用于历史 run 复现。

8.6 Contract Guard

`contract_guard.py` 是生成测试和 Reflection 的重要边界。

当前检查包括:

- 禁止 skip / xfail;
- 禁止 trivial assert;
- 禁止 broad except;
- 检查测试函数数量;
- 检查目标映射;
- 检查 FastAPI route 调用路径和方法;
- 检查 request field;
- 检查 response model field;

- 检查 source-backed oracle;
- 检查 Reflection 是否删除或削弱断言;
- 检查疑似业务代码修改倾向。

Contract Guard 不是提示词建议，而是 pipeline 边界。违反时应记录为 pipeline reject 或 invalid test set，而不是把失败算成 0 捕获。

9. Tool 设计

9.1 ToolRegistry

Tool 入口位于 `backend/app/tools/_init_.py`，`default_registry()` 注册默认工具。

主要工具：

工具	文件	职责
<code>list_files</code>	<code>fs_tools.py</code>	列出受控目录文件
<code>read_file</code>	<code>fs_tools.py</code>	读取受控文件
<code>analyze_project</code>	<code>analyze.py</code>	AST 分析项目结构、路由、函数、模型、已有测试
<code>write_test_file</code>	<code>write_test.py</code>	写入生成测试文件
<code>run_pytest</code>	<code>run_pytest.py</code>	执行 pytest 并解析结构化结果
<code>rg_search</code>	<code>search.py</code>	文本搜索
<code>ast_grep_search</code>	<code>search.py</code>	AST 模式搜索
<code>lsp_definition</code>	<code>lsp.py</code>	LSP/AST 定义跳转
<code>lsp_references</code>	<code>lsp.py</code>	LSP/AST 引用查询

9.2 ToolContext

ToolContext 包含：

- snapshot root;
- test write dir;
- pytest executor;
- 路径解析和白名单校验能力。

所有文件型工具必须通过 ToolContext 做路径规范化，防止越界访问。

9.3 analyze_project

`analyze_project` 使用 AST 提取：

- Python 文件;
- FastAPI route;
- function;
- Pydantic model;
- existing tests;
- fixture;
- dependency file;
- warnings。

分析范围来自 target scope。工具会避免把 generated test artifacts 作为 existing oracle 污染后续上下文。

9.4 write_test_file

`write_test_file` 只允许写入生成测试目录。

输出包括：

- file ID;
- path;
- content hash;
- bytes。

写入业务源码是禁止行为。

9.5 run_pytest

`run_pytest` 调用执行器运行 pytest，并转换成结构化 `RunPytestOutput`。

结果包括：

- exit code;
- collected、passed、failed、skipped;
- duration;
- case results;
- failures;
- artifacts;
- error。

case result 会分类 failure type, 并保存 stdout/stderr 摘要。

10. pytest 执行器设计

10.1 执行器端口

执行器协议 `PytestExecutor` 定义:

- `prepare_workspace()` ;
- `run_setup()` ;
- `run()` ;
- `collect_artifacts()` ;
- `cleanup_workspace()` ;
- `capabilities()` 。

这样 `run_pytest` 和 `replay` 可以依赖统一端口, 不直接绑定 local subprocess 或 Docker。

10.2 LocalSubprocessExecutor

默认执行器是 `LocalSubprocessExecutor` 。

特性:

- 使用本地 Python subprocess;
- 强制注入 `_trace_pytest_plugin` ;
- 强制 `--rootdir` 为 `work_dir`;
- 禁用 pytest cache;
- 屏蔽被测项目 addopts;
- 生成 `.trace_pytest_report.json` 和 `.trace_junit.xml` ;
- 支持硬超时;
- 返回 executor metadata。

明确限制:

- isolation level 是 process;
- 不强制禁网;
- 不强制资源限制;
- 不是安全沙箱。

10.3 DockerPytestExecutor

代码中已实现 `opt-in DockerPytestExecutor`。

特性：

- 使用 `docker run` 执行 `pytest`；
- `workspace bind mount` 到容器；
- `TRACE pytest plugin` 只读挂载；
- 可配置 `image`、`network policy`、`resource limits`、`working dir`、`pull policy`；
- 可在 `network disabled` 或 `install_only` 下控制运行阶段网络；
- 返回 `Docker command`、`image metadata`、`workspace mount` 和 `artifact paths`。

交付边界：

- `Docker executor` 不是默认演示路径；
- `Compose` 产品栈没有默认挂载 `Docker socket`；
- `Docker unavailable` 不进入捕获率分母。

10.4 pytest 插件

`backend/app/tools/_trace_pytest_plugin.py` 负责收集：

- `runtest logreport`；
- `collection report`；
- `collection finish`；
- `session finish`；
- `failure longrepr`。

执行器通过 `TRACE_PYTEST_REPORT` 环境变量指定 `JSON` 报告输出路径。

11. Reflection 详细设计

11.1 触发条件

Reflection 由策略和 `plan input` 共同控制：

- `strategy version` 允许 reflection；
- `test plan / budget` 允许 reflection；
- `pytest` 首轮执行未通过；

- failure 类型属于可修复测试问题;
- 未超过一次反思限制。

11.2 输入内容

Reflection 输入包括:

- 上一轮生成测试文件;
- pytest 结构化失败结果;
- source context;
- analysis summary;
- plan item;
- Reflection contract。

11.3 输出内容

Reflection 输出包括:

- fixed content;
- fix reason;
- changed points;
- lowered assertion strength;
- suspected code bug。

11.4 接受规则

系统只接受满足契约的 Reflection 结果:

- 测试仍有有效断言;
- 没有 skip / xfail;
- 没有 broad except;
- 没有删掉核心测试;
- 没有削弱断言;
- 没有修改业务源码。

如果不满足, 记录 rejected attempt 或 pipeline reject。

12. Report 设计

12.1 Run-level Report

agents/report.py 生成 run-level report。

内容包括：

- summary;
- metrics;
- risk notes;
- report quality;
- test inventory;
- target mappings;
- assertion summaries;
- failure classifications;
- reflection evidence;
- context completeness;
- Markdown report artifact;
- JSON report artifact。

12.2 Report Quality Evidence

report quality 用于支撑“测试是否有意义”：

- 测试清单;
- 目标映射;
- 断言摘要;
- 失败分类;
- Reflection 证据;
- 上下文完整性;
- source snippets;
- missing targets;
- risk notes。

这些信息会在 Run Console 和 Experiment metrics 中展示。

12.3 Artifact 记录

Artifact 只记录元数据：

- ID;
- run ID;
- attempt ID;

- artifact type;
- URI;
- content hash;
- size bytes;
- metadata。

大文件由本地文件系统存放。后续可迁移到对象存储，但目前未实现。

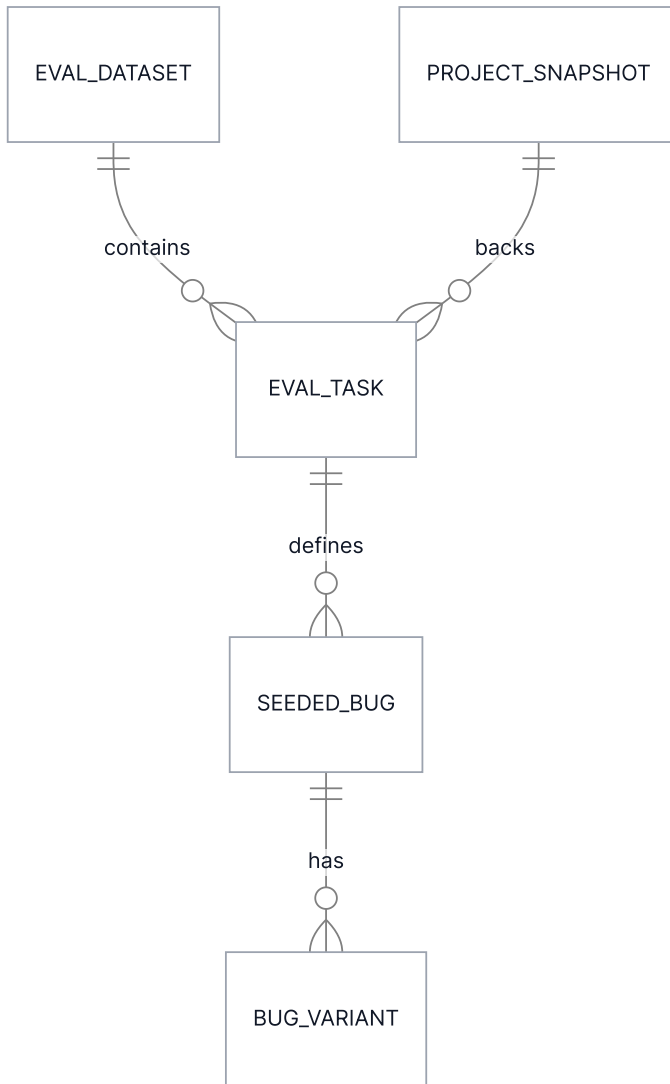
13. Evaluation Dataset 设计

13.1 Dataset 结构

Eval dataset 包含：

- dataset;
- eval task;
- seeded bug;
- bug variant;
- ground truth;
- project snapshot。

关系：



13.2 Readiness

Dataset readiness 是结构检查。

检查项包括：

- dataset 是否有 task;
- task 是否有 target scope;
- task 是否有 seeded bug;
- seeded bug 是否有 variant;
- variant 是否有 ground truth;
- auto mutation variant 是否有 probe check 且 passed。

readiness 不是 replay 结果，也不是捕获率。

13.3 Variant Canonical Patch

当前 bug variant 以 canonical patch 为主。

创建 variant 时：

- 1. 校验 patch file 不越界；
- 2. 校验 old content 在 clean snapshot 中唯一命中；
- 3. 如果给出 mutated snapshot，校验其内容与 patch 应用结果一致；
- 4. 将 patch artifact 内联保存到 ground truth；
- 5. 记录 patch unique hit 证据。

13.4 Mutation Discovery

Mutation discovery 流程：

- 1. 对 eval task 执行 dry-run；
- 2. 返回 mutation candidate、selection status 和 exclusions；
- 3. 前端选择 selected candidate；
- 4. confirm-selected 时后端重新 discovery；
- 5. 校验 audit report 与当前 server discovery 一致；
- 6. 校验 patch 唯一命中；
- 7. 在临时目录应用 patch；
- 8. 执行 probe check；
- 9. 创建 auto mutation seeded bug 和 variant。

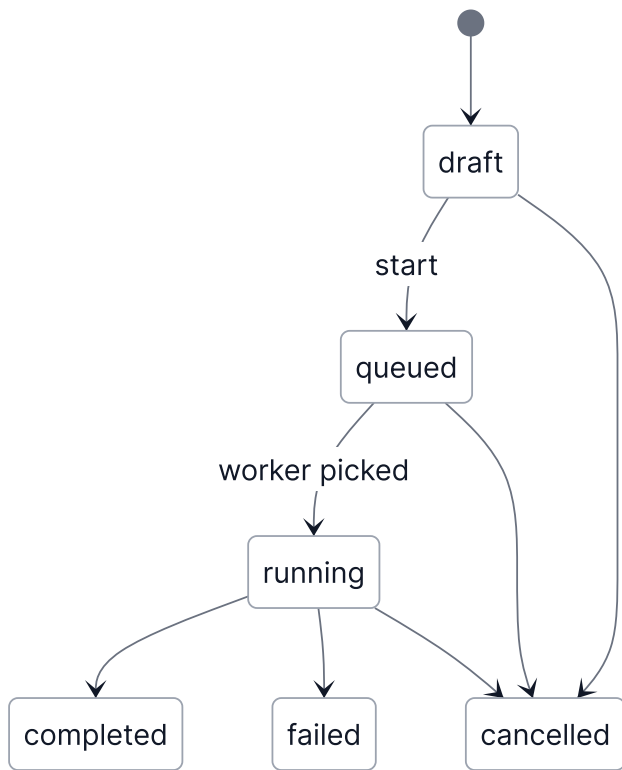
信任根在服务端，不在前端生成的 probe JSON。

14. Experiment 设计

14.1 Experiment 状态机

Experiment 状态：

状态	含义
draft	已创建，未启动
queued	已入队
running	Worker 正在执行
completed	实验完成
failed	实验失败
cancelled	用户取消



规则：

- 只有 draft 可原地启动；
- queued/running/completed/cancelled 再次 start 返回当前状态；
- failed 不允许原地 rerun；
- 新一轮评测应创建新 experiment。

14.2 创建设计

`create_experiment()` 校验：

- dataset 存在；
- runtime profile 存在且未归档；
- runtime profile 属于 dataset 项目；
- 多项目 dataset 必须提供完整 runtime bindings；
- strategy version 存在；
- strategy version 不重复；
- experiment ID 不冲突；
- llm override 可被 schema 校验。

创建后写入：

- `experiments` ；
- `experiment_strategy_versions` 。

14.3 执行主循环

`run_experiment()` 的逻辑维度：

- task;
- strategy version;
- repeat index。

每个 `(task, strategy, repeat)` 单元执行：

1. 选择 task 对应 snapshot;
2. 解析 runtime profile;
3. 创建 test plan;
4. 创建 clean run;
5. 做 source context gate;
6. 执行 clean run;
7. 提取 final test set;
8. 保存 generated test set artifact;
9. 保存 experiment clean run;
10. 执行 clean replay;
11. 执行 flaky gate;
12. 调度 variant replays;
13. 持久化 replay 和 capture 结果。

14.4 Context Gate

Context gate 用于防止 target scope 与 source context 不一致。

流程：

1. 对 snapshot root 调用 `analyze_project`;
2. 构造 source context bundle;
3. 检查 context completeness;
4. 若目标缺失，标记 run invalid;
5. 保存空 final test set artifact;
6. clean metrics 写入 `invalid_test_set`。

这类 invalid 不进入捕获率分母。

14.5 Final Test Set

Final test set 来自最终 attempt:

- 如果没有 Reflection, 取 initial attempt;
- 如果 Reflection 被接受, 取 reflection attempt;
- 参数化 pytest case 仍归并到生成测试函数;
- final set 保存文件内容、clean passed nodeids、case 数和 collection error 数。

Final test set 作为 artifact 冻结, replay 阶段只使用该 artifact。

14.6 Clean Replay 与 Flaky Gate

clean run 完成后还要执行 clean replay。

目的:

- 验证冻结测试集在干净代码上仍可复现;
- 避免生成环境和 replay 环境结果不一致;
- 在 variant replay 前先排除 flaky clean replay。

若 clean replay collection error、失败或 flaky gate 不稳定, 该单元标记 invalid_test_set, 不继续 variant replay。

14.7 Variant Replay

Variant replay 的输入:

- clean run;
- generated test set artifact;
- target snapshot;
- bug variant;
- clean passed nodeids;
- timeout;
- replay concurrency。

Variant replay 不调用 LLM。

捕获判定:

- replay 完成;
- pytest 收集成功;
- 至少一个 clean passed 测试在 variant 上 assertion failure;

- failure 与 variant ground truth 相关;
- captured_bug 写入 experiment_replay_runs 。

15. Replay 与缓存设计

15.1 Replay 工作目录

experiment workspace 根目录为 .pytest_tmp_experiments 。

每个 replay 会物化独立 workspace, 并在 manifest 中记录:

- workspace root;
- setup 状态;
- source replay;
- reused workspace;
- artifact 信息;
- executor metadata。

所有 cleanup 必须限制在 experiment workspace 根目录内。

15.2 Replay Cache

Replay cache 使用 cache key 记录相同 frozen test set 和 target snapshot 的执行结果。

状态:

- miss ;
- hit ;
- stale 。

cache hit 时可复用 source replay 的结果和 workspace 证据。清理时 source replay evidence 默认不删除。

15.3 Replay Scheduler

Replay scheduler 支持:

- variant replay concurrency;
- max retries;
- retry backoff seconds。

这些来自 runtime snapshot 的 replay policy。

15.4 Cleanup

workspace cleanup:

- 先 dry-run;
- 默认 keep_failed;
- 跳过 reused workspace;
- 校验路径必须在 experiment work root 内;
- 执行时删除候选目录。

replay cache cleanup:

- 先 dry-run;
- 跳过 source replay evidence;
- 只删除安全范围内 cache workspace;
- 不修改历史 metrics。

16. Metrics 设计

16.1 指标聚合入口

`experiment_metrics.py` 负责从数据库聚合 experiment metrics。

主要输入:

- experiment;
- experiment strategy versions;
- experiment clean runs;
- test runs;
- test reports;
- test replays;
- experiment replay runs;
- run artifacts;
- evaluation events。

16.2 MetricRun 中间结构

MetricRun 聚合一个策略重复下的主要数据:

- strategy ID;
- strategy name;
- repeat;
- status;
- clean failed;
- n cases;
- tokens;
- tool calls;
- reflection used;
- duration;
- collection success;
- reflection contract;
- invalid test set;
- variants capture map。

16.3 核心指标

输出指标包括：

- capture rate;
- false positive rate;
- cost per captured bug;
- average duration;
- pytest collection success rate;
- reflection contract pass rate;
- reflection acceptance rate;
- avg tokens;
- avg tool calls;
- metric status。

16.4 Capture Matrix

Capture matrix 以 bug variant 为行、strategy 为列。

同时提供：

- boolean matrix;
- count matrix;
- repeat count;
- per-variant capture rate。

16.5 Sampled Mutation Score

sampled mutation score 只统计：

- `ground_truth.source=auto_mutation` ；
- `ground_truth.probe.probe_check.status=passed` 。

ordinary variants 不进入该分母。probe failed 或 probe missing 的 auto mutation variant 也不进入分母。

16.6 Runtime Execution Summary

runtime execution summary 包含：

- executor kind distribution;
- runtime profiles;
- replay cache counts;
- setup status counts;
- replay concurrency;
- retries;
- observed/reused replay count;
- artifact inventory。

前端 Experiment detail 使用该结构展示 Runtime evidence。

16.7 Evaluation Events

Evaluation events 用于区分：

- invalid test set;
- provider failure;
- setup failure;
- executor failure;
- artifact mismatch;
- flaky clean replay;
- prompt-safe evidence;
- audit-only evidence;
- backfeed decision。

这些事件帮助前端避免把系统失败展示成 0 捕获。

17. Runtime Profile 设计

17.1 Profile 字段

Runtime profile 字段包括：

- project ID;
- name;
- executor;
- image;
- working dir;
- Python version;
- install command;
- test command;
- env template;
- resource limits;
- replay policy;
- network policy;
- artifact policy;
- cleanup policy;
- archived at.

17.2 校验规则

校验规则：

- executor 只能是 local_subprocess 或 docker ;
- network policy 只能是 default 、 disabled 、 install_only ;
- timeout 必须为正数;
- env template 不允许包含 secret;
- docker executor 必须有 image;
- local subprocess 不允许声称强制资源限制;
- local subprocess 不能配置非 default network policy;
- replay concurrency 至少为 1;
- max retries 和 retry backoff 不能为负。

17.3 Preflight

preflight 输出：

- status;
- checks;
- replay policy;
- audit-only limitations。

检查内容:

- profile contract;
- test command 解析;
- executor capabilities;
- local subprocess sandbox warning;
- docker availability;
- docker image warning。

preflight 不执行生成测试，只做结构和环境能力检查。

17.4 Executor Capabilities

capabilities 字段包括:

- executor kind;
- isolation level;
- network enforced;
- resource limits enforced;
- workspace strategy;
- supports parallel;
- policy notes。

这些字段会进入 runtime snapshot 和 metrics runtime execution。

18. LLM 设计

18.1 Provider 支持

当前支持:

- mock ;
- openai ;
- openai_chat_compat 。

实现位置：

- `llm.py` ；
- `llm_factory.py` ；
- `llm_config.py` ；
- `llm_openai.py` ；
- `llm_openai_chat_compat.py` ；
- `llm_http.py` 。

18.2 配置来源

LLM 配置可来自：

- 环境变量；
- `.env` ；
- `backend/llm.config.json` 。

真实 key 不写入 Git，不写入 snapshot，不在 API error 中回显。

18.3 MockLLM

MockLLM 用于：

- 后端测试；
- `compose smoke`；
- 稳定 benchmark；
- UI 演示。

MockLLM 是确定性链路验证，不代表真实模型效果。

18.4 HTTP 错误脱敏

`llm_http.py` 对 HTTP error 做脱敏：

- 不回显 API key；
- 不回显敏感 header；
- 避免把 provider 原始响应中可能包含的 secret 原样传出。

19. 前端设计

19.1 技术栈

前端使用：

- Vue 3;
- TypeScript;
- Vite;
- lucide-vue icons;
- hash route;
- 自定义 i18n。

19.2 应用入口

入口文件：

- frontend/src/main.ts
- frontend/src/App.vue

App.vue 负责：

- 顶部导航;
- hash route 解析;
- API/demo 数据源切换;
- 中文/英文切换;
- 页面组件分发。

19.3 页面设计

页面包括：

页面	文件	职责
ProjectListPage	pages/ProjectListPage.vue	项目列表、注册项目、创建 snapshot、创建 run
RunConsolePage	pages/RunConsolePage.vue	run 概览、trace、report、pytest、artifacts
ExperimentListPage	pages/ExperimentListPage.vue	experiment 列表和创建表单
ExperimentDetailPage	pages/ExperimentDetailPage.vue	start、progress、metrics、evidence、cleanup、export
DatasetListPage	pages/DatasetListPage.vue	dataset 列表和创建

页面	文件	职责
DatasetDetailPage	pages/DatasetDetailPage.vue	readiness、task、bug、variant、mutation discovery、创建 experiment
RuntimeProfilePage	pages/RuntimeProfilePage.vue	runtime profile、executor status、preflight
ComparisonPage	pages/ComparisonPage.vue	completed experiment 策略对比

19.4 API Client

API client 位于 frontend/src/api 。

按业务拆分：

- projects;
- runs;
- experiments;
- evaluation;
- runtime profiles;
- strategies;
- local;
- llm options。

所有 API 请求经 client.ts 统一处理。

19.5 组件设计

通用组件包括：

- TraceTimeline;
- TraceStepDetail;
- RunSummaryPanel;
- ReportPaper;
- PytestResultsTable;
- ArtifactsList;
- JsonViewer;
- StatusBadge;
- StageRail。

Experiment 专用组件包括：

- ExperimentMetricsTable;
- CaptureMatrix;
- EvaluationEventTimeline;
- ReplayEvidencePanel;
- CleanRunTable。

19.6 数据源设计

前端支持两种数据源：

- API;
- demo。

规则：

- API 模式用于真实验收；
- demo 模式只用于静态前端预览；
- Comparison API 模式不回退 demo metrics；
- 从 demo 切回 API 时，示例 run/experiment/dataset 路由会映射回真实入口。

19.7 i 18 n 设计

frontend/src/i18n.ts 内置 zh/en 字典。

语言选择写入 localStorage，页面通过 useI18n() 获取 t() 。

20. API 接口设计

20.1 项目接口

方法	路径	作用
POST	/api/v1/projects	创建项目
GET	/api/v1/projects	项目列表
GET	/api/v1/projects/{project_id}	项目详情
POST	/api/v1/projects/{project_id}/snapshots	创建快照
GET	/api/v1/projects/{project_id}/snapshots	快照列表
GET	/api/v1/projects/{project_id}/test-plans	项目计划列表
GET	/api/v1/projects/{project_id}/test-runs	项目运行列表

20.2 Run 接口

方法	路径	作用
POST	/api/v1/test-plans	创建测试计划
GET	/api/v1/test-plans/{plan_id}	测试计划详情
GET	/api/v1/test-plans/{plan_id}/runs	计划运行列表
POST	/api/v1/test-plans/{plan_id}/runs	创建并入队 run
GET	/api/v1/test-runs/{run_id}	run 详情
POST	/api/v1/test-runs/{run_id}/cancel	取消 run
POST	/api/v1/test-runs/{run_id}/retry	重试 run
GET	/api/v1/test-runs/{run_id}/trace-steps	trace steps
GET	/api/v1/test-runs/{run_id}/events	run events
GET	/api/v1/test-runs/{run_id}/attempts	attempts
GET	/api/v1/test-runs/{run_id}/pytest-results	pytest results
GET	/api/v1/test-runs/{run_id}/artifacts	artifacts
GET	/api/v1/test-runs/{run_id}/report	report
PATCH	/api/v1/generated-test-cases/{case_id}/adoption	更新用例采纳状态

20.3 Versioning 接口

方法	路径	作用
POST	/api/v1/prompt-versions	创建 prompt version
GET	/api/v1/prompt-versions/{id}	查询 prompt version
POST	/api/v1/tool-schema-versions	创建 tool schema version
GET	/api/v1/tool-schema-versions/{id}	查询 tool schema version
POST	/api/v1/strategies	创建 strategy
GET	/api/v1/strategies	strategy 列表
POST	/api/v1/strategies/{id}/versions	创建 strategy version
GET	/api/v1/strategies/{id}/versions	strategy versions
GET	/api/v1/strategy-versions/{id}	strategy version 详情

20.4 Runtime 接口

方法	路径	作用
POST	/api/v1/projects/{project_id}/runtime-profiles	创建项目 runtime profile
POST	/api/v1/eval-datasets/{dataset_id}/runtime-profiles	为 dataset 创建 runtime profile
GET	/api/v1/projects/{project_id}/runtime-profiles	项目 runtime profiles

方法	路径	作用
GET	/api/v1/eval-datasets/{dataset_id}/runtime-profiles	dataset runtime profiles
GET	/api/v1/eval-datasets/{dataset_id}/runtime-binding-manifest	多项目 runtime binding manifest
GET	/api/v1/runtime-profiles/executors/status	executor status
POST	/api/v1/runtime-profiles/preflight	payload preflight
POST	/api/v1/runtime-profiles/{id}/preflight	persisted profile preflight
GET	/api/v1/runtime-profiles/{id}	runtime profile 详情
PATCH	/api/v1/runtime-profiles/{id}	更新 runtime profile
POST	/api/v1/runtime-profiles/{id}/archive	归档 runtime profile

20.5 Evaluation 接口

方法	路径	作用
POST	/api/v1/eval-datasets	创建 dataset
GET	/api/v1/eval-datasets	dataset 列表
GET	/api/v1/eval-datasets/{id}	dataset 详情
GET	/api/v1/eval-datasets/{id}/readiness	readiness
POST	/api/v1/eval-datasets/{id}/tasks	创建 task
PATCH	/api/v1/eval-tasks/{id}	更新 task
GET	/api/v1/eval-datasets/{id}/tasks	task 列表
POST	/api/v1/eval-tasks/{id}/seeded-bugs	创建 seeded bug
PATCH	/api/v1/seeded-bugs/{id}	更新 seeded bug
GET	/api/v1/eval-tasks/{id}/seeded-bugs	seeded bug 列表
POST	/api/v1/eval-tasks/{id}/mutation-discovery/dry-run	mutation discovery dry-run
POST	/api/v1/eval-tasks/{id}/mutation-discovery/confirm-selected	确认 selected mutation
POST	/api/v1/seeded-bugs/{id}/variants	创建 variant
PATCH	/api/v1/bug-variants/{id}	更新 variant
GET	/api/v1/seeded-bugs/{id}/variants	variant 列表

20.6 Experiment 接口

方法	路径	作用
POST	/api/v1/experiments	创建 experiment
GET	/api/v1/experiments	experiment 列表

方法	路径	作用
GET	/api/v1/experiments/{id}	experiment 详情
POST	/api/v1/experiments/{id}/runs	启动 experiment
POST	/api/v1/experiments/{id}/cancel	取消 experiment
GET	/api/v1/experiments/{id}/clean-runs	clean runs
GET	/api/v1/experiments/{id}/replay-runs	replay runs
GET	/api/v1/experiments/{id}/metrics	metrics
GET	/api/v1/experiments/{id}/progress	progress
POST	/api/v1/experiments/{id}/cleanup	workspace cleanup
GET	/api/v1/experiments/{id}/artifact-inventory	artifact inventory
GET	/api/v1/experiments/{id}/replay-cache	replay cache
POST	/api/v1/experiments/{id}/replay-cache/cleanup	replay cache cleanup

21. 错误处理设计

21.1 错误分类

系统错误主要分为：

- 输入错误；
- 资源不存在；
- 状态冲突；
- LLM provider 错误；
- 工具参数错误；
- 项目分析失败；
- pytest collection 失败；
- pytest execution 失败；
- runner/internal 错误；
- reflection contract violation；
- artifact 错误；
- replay/cache/cleanup 错误。

21.2 API 错误映射

Service 层异常带有 status code 语义：

- not found → 404；

- conflict → 409;
- validation/value error → 400;
- unexpected → 500。

experiments 路由中特别注意: `ExperimentError` 必须先于普通 `ValueError` 捕获, 避免 409 被错误映射成 400。

21.3 Run 失败写入顺序

run 失败处理顺序:

1. rollback 当前事务;
2. 重新读取 run;
3. 如果已 completed/failed/cancelled 则不覆盖;
4. 写入 run event;
5. 设置 status failed;
6. 设置 error code/message;
7. 设置 finished_at;
8. commit。

Agent orchestrator 内部也遵守: 先 event/trace, 再改 status。

21.4 Experiment 失败处理

`run_experiment()` 外层捕获异常:

- rollback;
- 重新读取 experiment;
- 设置 status failed;
- 写入 error code 和 error message;
- 设置 finished_at;
- commit;
- 继续抛出给 worker 记录日志。

cancelled 使用单独 `ExperimentCancelledError` 分支, 最终状态为 cancelled。

22. 安全设计

22.1 路径安全

路径安全由多层实现：

- snapshot root 必须在 project root 内；
- generated tests dir 由系统创建；
- ToolContext 做 safe resolve；
- read/write/search 工具限制在 root 内；
- write_test_file 只写 generated tests；
- cleanup path 必须在 experiment work root 内；
- cleanup 禁止操作 experiment work root 本身。

22.2 Secret 安全

secret 安全规则：

- env template 创建时拒绝 secret；
- runtime snapshot 只保存 env keys 和脱敏 env template；
- LLM API key 不写入 strategy snapshot；
- LLM config error 不回显 secret；
- .env 和 backend/llm.config.json 不提交 Git。

22.3 执行安全

当前执行安全边界：

- local subprocess 有路径校验和超时；
- 不是安全沙箱；
- Docker executor 是 opt-in；
- Compose 不挂 Docker socket；
- 不建议对不可信项目执行生成测试。

22.4 Agent 安全

Agent 安全边界：

- 只能通过 ToolRegistry 调用工具；
- 不直接执行任意 shell；
- 不允许写业务源码；
- Contract Guard 拦截危险或无意义测试；
- Reflection 禁止削弱断言。

23. 可观测性设计

23.1 Trace Step

Trace step 保存:

- run ID;
- attempt ID;
- step index;
- step type;
- name;
- input summary;
- output summary;
- tool name;
- payload;
- tokens;
- duration;
- status;
- error。

前端 Run Console 通过 trace timeline 展示。

23.2 Run Event

Run event 保存状态机事件:

- run ID;
- stage;
- event type;
- status before;
- status after;
- message;
- created at。

Run event 用于审计状态变化，不替代 trace。

23.3 Evaluation Event

Evaluation event 在 experiment metrics 中投影，帮助解释:

- 哪些失败可进 prompt;
- 哪些只做 audit;
- 哪些事件阻断指标;
- 哪些 failure 不属于 0 捕获。

23.4 Runtime Evidence

Runtime evidence 包括:

- executor distribution;
- runtime profile;
- setup status;
- replay cache;
- retry;
- artifact inventory;
- replay llm_calls。

这些证据用于验收时证明 replay 没有重新调用 LLM。

24. 测试与验证设计

24.1 后端测试

后端测试位于 `backend/tests` 。

覆盖范围包括:

- API async run;
- API retry/cancel;
- DB models;
- schema migrations;
- strategy versioning;
- runtime profiles;
- LLM options;
- source context;
- Contract Guard;
- pytest executor;
- replay scheduler;
- artifact cleanup;

- evaluation seed;
- experiment runner;
- eval harness。

24.2 前端构建

前端通过 `npm run build` 验证 TypeScript 和 Vite 构建。

24.3 Compose Smoke

`scripts/compose_smoke.ps1` 验证：

- `/healthz` ;
- LLM options;
- dataset seed;
- 前端 app shell;
- 可选最小 experiment run。

24.4 Compose Benchmark

`scripts/compose_benchmark.ps1` 验证：

- Direct / Plan / ReAct 三策略;
- dataset-demo-v 2;
- task/variant 数动态匹配;
- metric status;
- replay `llm_calls=0` ;
- metrics JSON 输出。

25. 交付运行设计

25.1 默认演示链路

默认演示链路：

```
Docker Compose 启动
→ 前端 API 模式
→ Dataset list
→ Dataset detail
→ Create Experiment
→ Draft detail
```

- Start
- Progress
- Completed Metrics / Evidence
- Report / Export
- Comparison

25.2 Demo Dataset

默认数据集：

- ID: dataset-demo-v2 ;
- task: 3 个;
- bug variants: 16 个;
- strategy: sv-direct-v1、sv-plan-v1、sv-react-v1 。

25.3 验收参考结果

已验证 benchmark:

- experiment: compose-benchmark-suite-r3-20260622112519 ;
- repeat: 3;
- clean runs: 27;
- variant replay records: 144;
- replay llm calls: [0] ;
- metric status: 三行 ok 。

参考指标：

Strategy	capture_rate_mean	false_positive_rate
Direct v 1	0.875	1.0
Plan-and-Execute v 1	1.0	0.0
ReAct + Reflection v 1	1.0	0.0

该结果来自 MockLLM，不与真实 LLM 混合解释。

26. 已知限制与后续设计

26.1 已知限制

- 当前主要面向 Python/FastAPI + pytest;

- local subprocess 不是安全沙箱;
- Docker executor 非默认交付;
- artifact 仍存本地文件系统;
- cancel 是软取消;
- experiment failed/cancelled 不支持原地 rerun;
- experiment-level Markdown report 未实现;
- 真实 LLM 效果依赖外部模型服务。

26.2 后续演进

后续可扩展:

- 远程 Runner 注册、心跳和 long-poll;
- Runner job lease 和幂等 result callback;
- Docker executor 默认化和容器加固;
- MinIO/S3 artifact storage;
- 完整 OpenTelemetry;
- experiment clone rerun;
- 多语言测试生成;
- 更完善的人工评分与采纳 workflow;
- 更丰富 mutation operator。

27. 结论

TRACE 当前详细设计可以概括为: API 管资源, Worker 管执行, Agent 管生成, Tool 管边界, PostgreSQL 管事实, 前端管证据展示。普通 run 解决“测试能否生成并执行”, experiment 解决“这些测试能否证明策略有效”。系统最重要的设计不是多复杂的框架, 而是几个硬边界: run 级快照冻结、受控工具、真实 pytest、一次受限 Reflection、frozen test set replay、指标口径不造假。

这套设计已经支撑交付演示和 benchmark 验收; 后续可以在 Runner、存储、观测和安全隔离上继续加厚, 但不能破坏当前这些已经站稳的业务边界。